Deutsche Bank

DAY 10 · 20-MINUTE DEMO

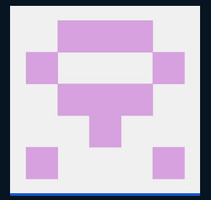
ReconX — Enterprise Trade Reconciliation Platform

Deutsche Bank TDI 2026 training case study

Contributors

VERIFIED PUBLIC GITHUB ROSTER · 7

Tobias Becher
@TB-DevAcc



gb-dev04
@gb-dev04

NJ Bodam
@NJBodam

Timur Garipov
@SaltyRain

anushadagar1407
@anushadagar1407

Aqib Khan
@Aqibkhan2023

Beyda
@beydarb

Evidence boundary first. Demonstration detail second.

TRAINING CASE STUDY · NOT PRODUCTION DEPLOYMENT

02 / OPERATIONS PROBLEM

Mismatches need a controlled path

One trade, two records, one break to investigate.

ReconX models the operator's journey from capture to comparison to evidence-backed resolution.

- Capture the internal trade record.
- Compare it with an external record.
- Investigate and resolve the break with evidence.

ILLUSTRATIVE DISCREPANCY · NOT PRODUCTION DATA

FIELD	INTERNAL RECORD	EXTERNAL RECORD
`trade reference`	same reference	same reference
`quantity`	captured	received
`price`	**recorded value**	**received value**

BREAK

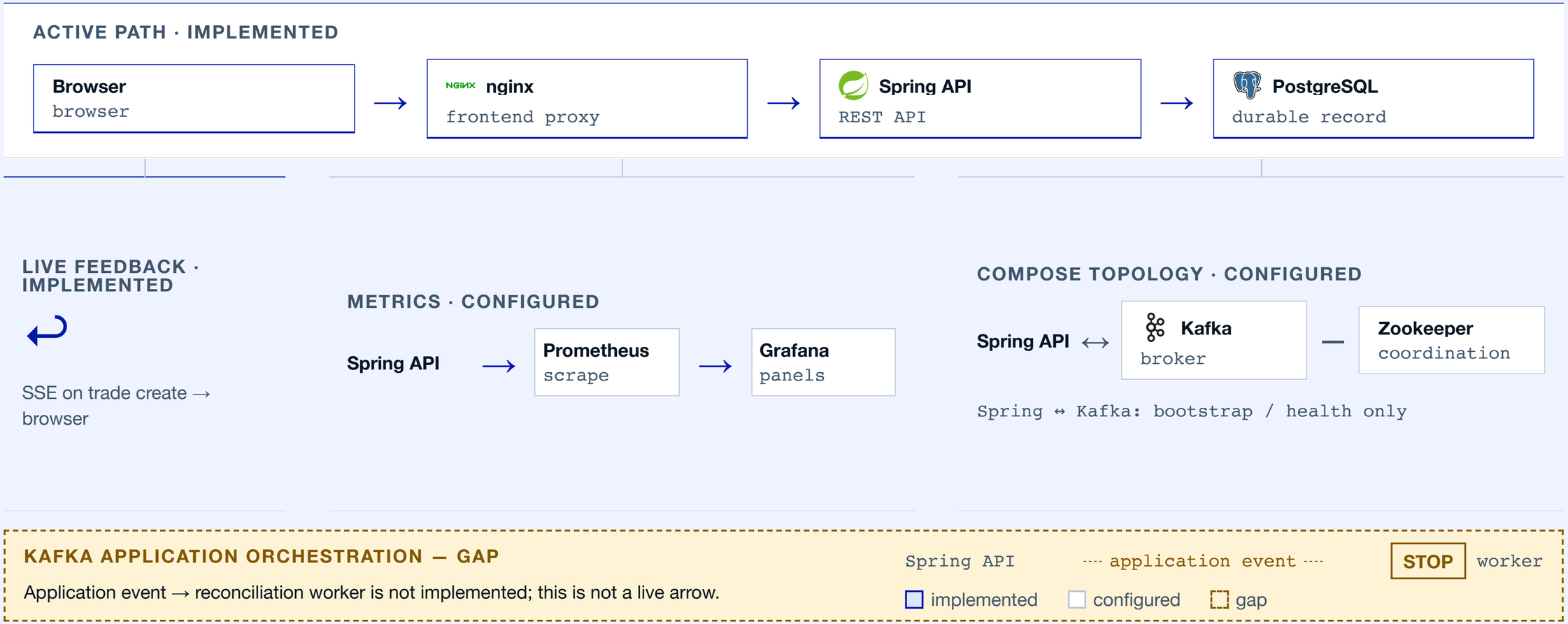
A mismatch is now inspectable: compare the records, then resolve with evidence.

03 / CURRENT RUNTIME BOUNDARY

SOURCE-SUPPORTED PATH

Architecture, as it runs today

Capture and observation are source-supported; event orchestration remains unfinished.



04 / STACK BY RESPONSIBILITY

A FOUNDATION WITH EXPLICIT MATURITY BOUNDARIES

Stack by responsibility, not by shopping list

01	DOMAIN / API	IMPLEMENTED	Java 25	Spring Boot 3.5.0	sealed types · DTO boundaries	
02	DATA / AUDIT	IMPLEMENTED	PostgreSQL 16	Liquibase · JPA / Envers	JSONB · partitions	
03	UI / LIVE UPDATES	IMPLEMENTED	React 19	Vite 6	nginx	SSE
04	MESSAGING / OPS	• CONFIGURED	Kafka scaffold	Prometheus 2.54.1	Grafana 11.2.0	application path unfinished
05	DELIVERY / PRESENTATION	• CONFIGURED	Docker	Compose	GitHub Actions verification	Reveal.js 6 · Playwright

Messaging is a configured dependency, not yet an integrated application path. Release status requires separate evidence; version strings alone are not release proof.

05 / LIVE TRADE JOURNEY

4:00

Live demo runway

Login → validated trade → live update. The slide stays useful if the app does not.

01 **Login**
JWT + role check

02 **Create**
validated POST

03 **Persist**
HTTP 201 → PostgreSQL

04 **Observe**
create-time SSE → trade_created_total

RUNTIME CAPTURE PENDING Launch the guarded frame when ready / If unavailable: full-demo link or source/API walkthrough

Demo: idle

Launch embedded demo

Open full demo

GUARDED LIVE WINDOW · EXPORT-SAFE IDLE STATE

Activate the frame only when the presenter is ready.

No runtime capture is claimed here. The full-demo link and source/API walkthrough remain available.

Reconciliation truth line

Core logic is real. The seam is visible.

INPUTS

Internal + external records

indexed by trade reference

→

IMPLEMENTED CORE

ReconciliationEngine

rule comparison + duration timer

→

OUTCOMES

MATCHED / BREAK

direct engine result

REST BOUNDARY

`POST /api/v1/recon/run`

202 · QUEUED

job row persisted

WORKER BOUNDARY

CONFIGURED ONLY

Apache Kafka

broker / health path

Not observed

Queue has no worker; no application listener result is observed.

07 / CI/CD DELIVERY RAIL · 11:00–12:40

CI verifies the build — then stops

A commit-bound verification rail, not a release claim.

VERIFIED REMOTE RUN

`PR #272 · run 30898100384`

`commit 4f1a939a`

TRIGGER

PR / manual dispatch

verification path

Backend

container / Compose

Frontend

container / Compose

Presentation

container / build

BUILD BOUNDARY

`push: false`

build completed · not pushed, loaded, or exported

NOT EVIDENCED HERE

GHCR · deployment · release

no success arrow

ARTIFACT SPOOL

JUnit / Failsafe / Vitest / JaCoCo reports

uploaded by full verification jobs

Passed on the named run

Backend, frontend, and presentation verification passed; JUnit/Vitest report checks passed.
Image builds completed without push, load, or export. Load job and native fallbacks: skipped.

08 / MONITORING & LOAD EVIDENCE · 12:40–14:25

Monitoring is provisioned; load proof is pending

The source has a dashboard. The run still needs to be captured.

EVIDENCE PATH

`source → runtime → recovery`

01

Five panels are defined.

+ request rate
+ P95 latency
+ open breaks
+ trades / second
+ reconciliation duration

`reconx-backend` scrapes `/api/actuator/prometheus`.

Invalid in-container scrape: `localhost:8080`.

02

No current image.

runtime capture pending

target state + panel values require a fresh run

A dashboard definition is not a measured screenshot.

03

The profile is on disk.

`10 VUs × 10 iterations = 100 trade creations`

Zero-failure checks are configured; no completed run is currently proven.

Next capture

200-VU baseline / load / recovery

PENDING

Configured now · fresh runtime evidence required

Keep the current 10-VU / 100-create truth. Do not promote a 200-VU success, latency SLA, or recovery claim without a captured run and provenance.

09 / AFTER-ACTION NOTES

1:35 · reflection before the close

Learnings we can defend

01

Configuration is not integration.

Promote a path only after its behavior is observable end to end.

02

Contract drift breaks rehearsals.

Keep the current request shape beside the walkthrough.

03

Evidence needs provenance.

Attach source, run, and capture context to every claim.

DELIVERY CHECK

Presenter reflection pending

Personalize with confirmed sentences before delivery. Keep the prompts team-level until then.

`No attribution · no rehearsal outcome · no invented voice`

10 / OPEN FLOOR

Questions?

Follow the evidence to the layer that needs a closer look.

ROUTE THE QUESTION

01 **API**
Trade boundary and request contract

02 **Recon**
Matching core and orchestration seam

03 **Runtime**
Compose path and observability

04 **Delivery**
Verification and source provenance

REPOSITORY / SOURCE

https://github.com/anushadagar1407/db2026-reconX

OPEN ANCHORS `TradeController` `ReconciliationEngine` `docker-compose.yml`

A sourced answer is better than a finished-sounding one.